

Aim

To study and perform **Version Control using Git & GitHub** by creating a local repository, tracking files, committing changes, and pushing the repository to GitHub.

Theory

Git is a distributed version control system used to track changes in source code and collaborate efficiently. It allows developers to maintain different versions of a project and restore previous versions whenever required.

GitHub is a cloud-based platform that hosts Git repositories online. It provides collaboration features such as pull requests, issue tracking, and remote repository management.

Version control helps developers:

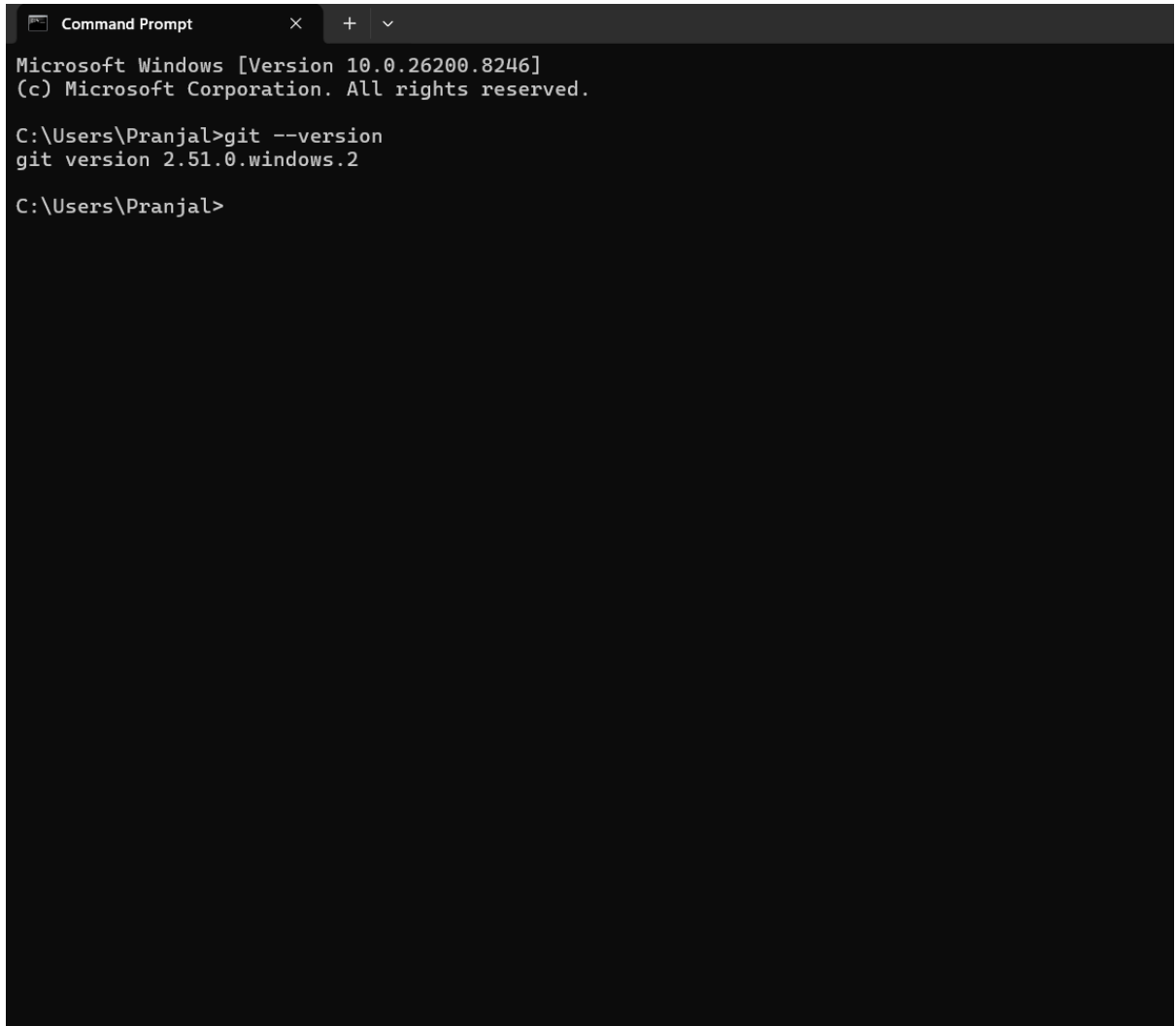
- Track changes in files
- Collaborate with multiple users
- Maintain project history
- Recover previous versions
- Manage branches and updates efficiently

Step by Step Procedure

1. Installation of Git
2. Configure username & email
3. Create local repository
4. Create files
5. Add files to staging
6. Commit changes

7. Create GitHub repository
8. Link local repo to GitHub
9. Push files

Screenshots -

A screenshot of a Windows Command Prompt window. The title bar at the top reads "Command Prompt" with standard window controls. The main area of the window is black with white text. It displays the Windows version information: "Microsoft Windows [Version 10.0.26200.8246]" and "(c) Microsoft Corporation. All rights reserved." Below this, the command prompt shows the user's location as "C:\Users\Pranjal>". The user has entered the command "git --version", and the output is "git version 2.51.0.windows.2". The prompt is now waiting for the next command.

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Pranjal>git --version
git version 2.51.0.windows.2

C:\Users\Pranjal>
```

1. Git installation verification (git version)

New repository

Q Type [Z] to search

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

pranjalsomatkar24-eng

Repository name *

My new repo

Your new repository will be created as My-new-repo. The repository name can only contain ASCII letters, digits, and the characters ., -, and _.

Great repository names are short and memorable. How about [vigilant-doodle?](#)

Description

local

5 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

2. Repository creation

Import favorites

McAfee Security

Agoda

Adobe offers

Dropbox Offer

pranjalsomatkar24-eng / My-new-repo

Q Type [Z] to search

<> Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

My-new-repo /

Drag additional files here to add them to your repository

Or [choose your files](#)

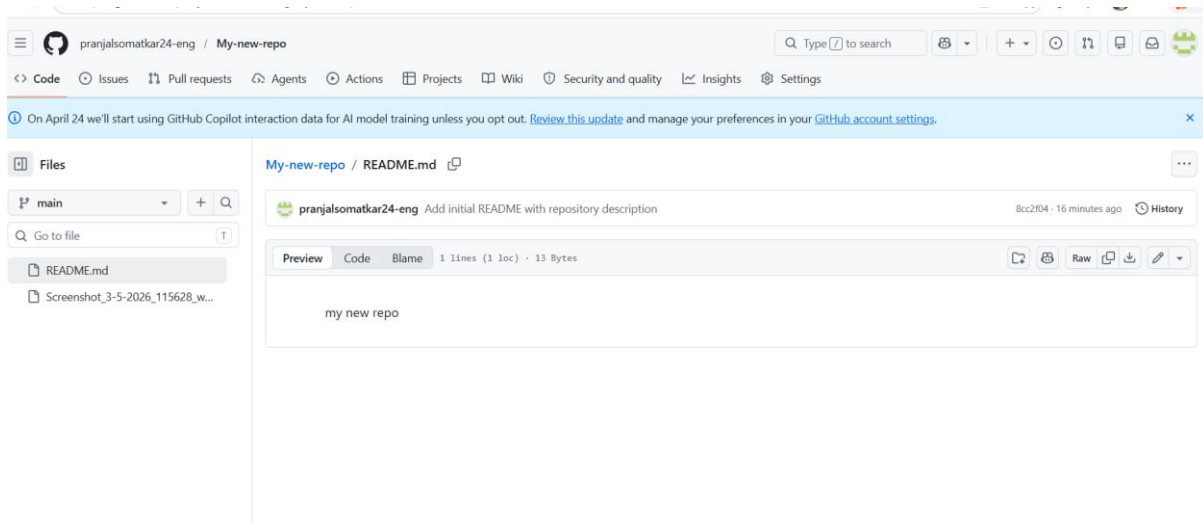
Screenshot_3-5-2026_115628_www.medicoverhospitals.in.jpeg

Commit changes

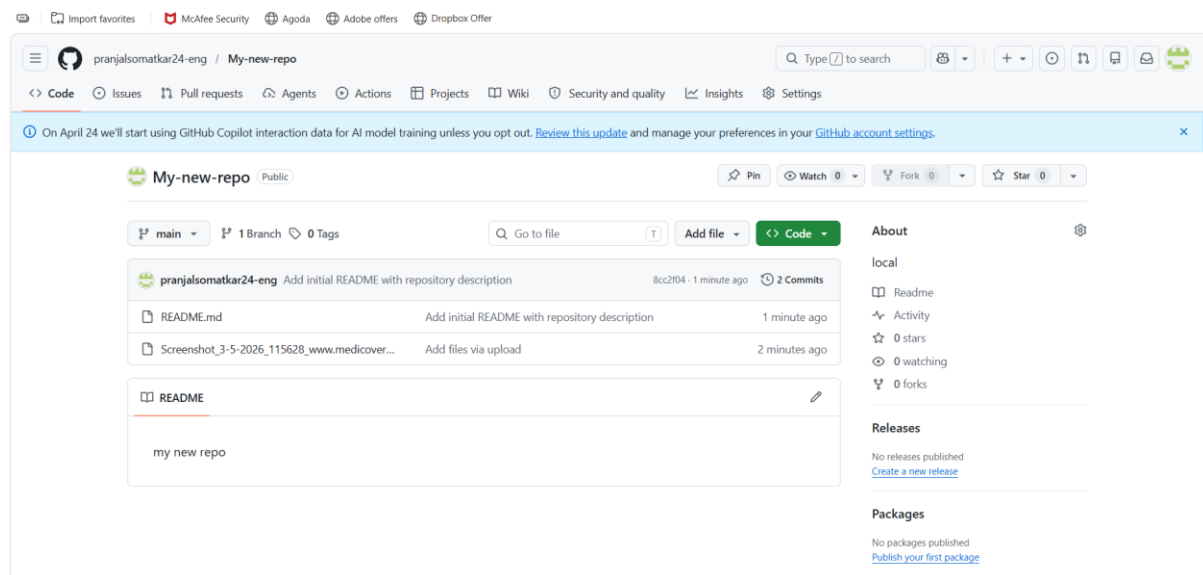
Add files via upload

Add an optional extended description...

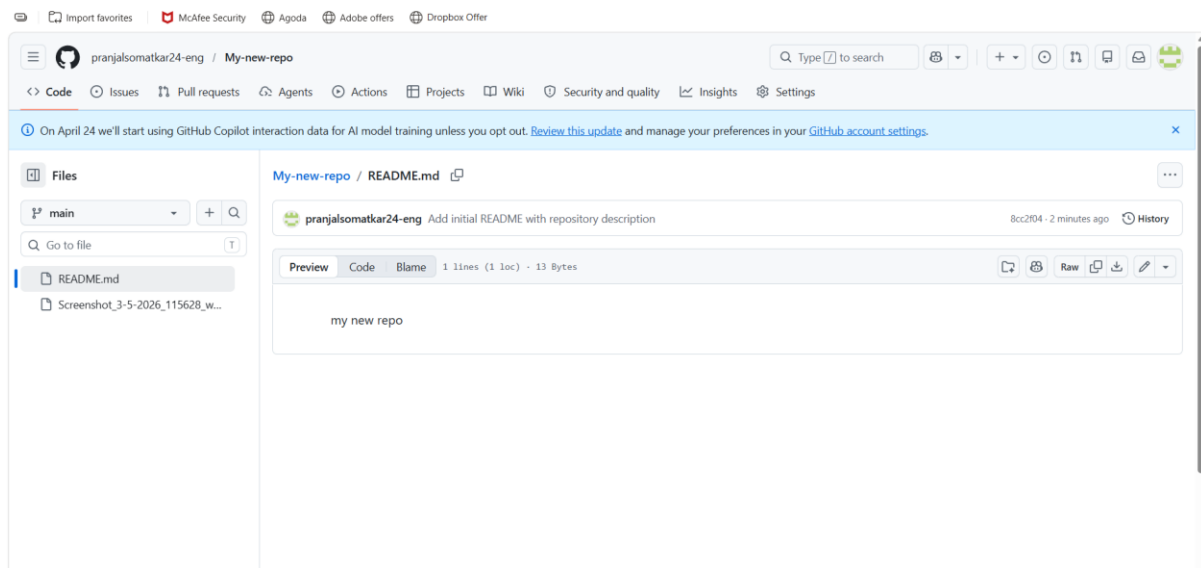
3. Commit history



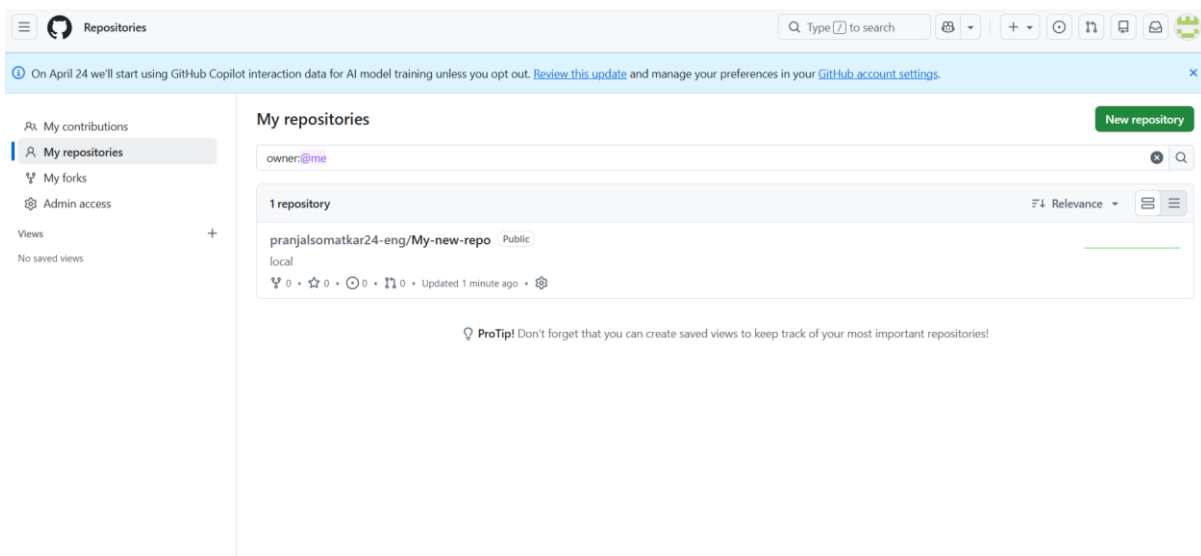
4. GitHub repository page



5. Successful push



6. Display commit logs (git log)



7. Show files on GitHub repo

Questions –

1. What is the difference between Git and GitHub?

Git

Git is a distributed version control system.

Used to track changes in files locally.

Works on a local machine.

Example commands: git add, git commit

GitHub

GitHub is a cloud-based platform for hosting Git repositories.

Used for online storage and collaboration.

Works on the internet/remotely.

Provides features like pull requests, issues, and repository sharing.

2. Explain the difference between git add and git commit

git add

Adds files/changes to the staging area.

Prepares files for commit.

Does not save changes permanently.

Example:

git

git commit -m "Added file"

git commit

Saves staged changes permanently to repository history.

Creates a snapshot/version of the project.

Permanently records changes with a message.

add

file.txt

3. What is the staging area in Git?

The **staging area** is an intermediate area where changes are prepared before committing them to the repository. It allows users to select specific changes or files that should be included in the next commit.

4. What happens if you modify a file after committing it?

If a file is modified after committing:

- Git marks the file as **modified**.
- The new changes are not included in the previous commit.
- You must run `git add` and `git commit` again to save the updated changes.

Check status using:

`git status`

5. Difference between git push and git pull

git push

Uploads local commits to a remote repository.

Local → Remote

Used to share changes.

git pull

Downloads updates from the remote repository to local system.

Remote → Local

Used to get latest changes.

6. What is Version Control? Differentiate between Centralized and Distributed VCS.

Version Control

Version Control is a system that tracks changes made to files over time, allowing users to manage different versions and restore previous versions if needed.

7. Centralized VCS vs Distributed VCS

Centralized VCS

Has one central server.

Requires network connection to access repository.

Slower and dependent on server.

Example: SVN, CVS

Distributed VCS

Every user has a complete repository copy.

Can work offline.

Faster and more flexible.

Example: Git, Mercurial